

VerifyAI Research Assistant

API Documentation · app.py & db.py

This document describes every HTTP endpoint exposed by the Flask backend (app.py), the authentication scheme, request/response shapes, and the PostgreSQL schema and query helpers defined in db.py. It is intended for frontend developers integrating against this API and for anyone onboarding to the backend.

Contents

- 1. Overview & Architecture
- 2. Environment Variables
- 3. Authentication
- 4. Endpoints
 - 4.1 GET /
 - 4.2 POST /register
 - 4.3 POST /login
 - 4.4 POST /predict/article
 - 4.5 POST /predict/url
 - 4.6 POST /predict/quote
 - 4.7 GET /history
- 5. Response Object Reference
- 6. Database Schema (db.py)
- 7. Error Handling Conventions

1. Overview & Architecture

The backend is a Flask application that combines a local machine-learning classifier (scikit-learn, trained on the ISOT dataset) with three external services:

- Groq (Llama 3.1) — generates short titles for predictions, used for history labels and as NewsAPI search queries.
- NewsAPI — fetches related/verified articles for the research dashboard, with auto-generated APA citations.
- PostgreSQL — stores user accounts and a log of every prediction made (via db.py).

All prediction endpoints are protected by JWT authentication. Tokens are issued by /login and must be sent as a Bearer token in the Authorization header on every protected request.

Request flow for an article prediction

- 1. Client sends article text or a URL to /predict/article or /predict/url.
- 2. Server fetches/cleans the text (trafilatura for URLs, strip_source_prefix for Reuters-style bylines).
- 3. TF-IDF vectorizer transforms the text; the ISOT model returns a REAL/FAKE verdict and confidence.
- 4. The prediction is logged to PostgreSQL (non-blocking — failures are caught and logged, not raised).
- 5. A linguistic dashboard (sentiment, readability, surface stats) is computed locally via TextBlob and textstat.
- 6. Groq generates a short title from the article; that title is used to query NewsAPI for related articles.
- 7. Related articles are returned with an excerpt (best-matching sentence) and an APA 7 citation.

2. Environment Variables

Loaded via python-dotenv from a .env file at the project root. All are required for full functionality; the app degrades gracefully (with warnings) if optional keys are missing.

Variable	Purpose
JWT_SECRET	Secret key used to sign and verify JWT tokens. Required.
GROQ_API_KEY	API key for Groq (Llama 3.1-8b-instant). Optional — if absent, titles fall back to top TF-IDF keywords.
NEWS_API_KEY	API key for NewsAPI.org. Optional — if absent, related_articles is returned as an empty list.

DB_HOST	PostgreSQL host.
DB_PORT	PostgreSQL port.
DB_NAME	PostgreSQL database name.
DB_USER	PostgreSQL username.
DB_PASSWORD	PostgreSQL password.

3. Authentication

Authentication uses JSON Web Tokens (JWT), signed with HS256 using JWT_SECRET.

Token lifecycle

- 1. User registers via POST /register (username, email, password). Password is hashed with bcrypt before storage.
- 2. User logs in via POST /login (username, password). On success, the server returns a signed JWT.
- 3. The token payload contains user_id, username, and an exp claim (expires 1 hour after issue).
- 4. The client includes the token on every protected request as: Authorization: Bearer <token>

token_required decorator

All prediction endpoints and /history are wrapped with @token_required, which:

- Reads the Authorization header and extracts the token after 'Bearer '.
- Decodes and verifies the token with jwt.decode(token, JWT_SECRET, algorithms=['HS256']).
- Attaches the decoded payload to request.user for use inside the route (e.g. request.user['user_id']).
- Returns 401 'Token is missing!' if no Authorization header is present.
- Returns 401 'Token is invalid!' if decoding/verification fails (expired, malformed, wrong signature, etc.).
- OPTIONS requests (CORS preflight) bypass the check entirely.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

4. Endpoints

Base URL during local development: `http://localhost:5000`

4.1 Health Check

GET /

Authentication: Not required

Simple liveness check. Returns a static JSON payload confirming the API is running.

Response — 200 OK

```
{
  "status": "ok",
  "message": "VerifyAI Research Assistant API running."
}
```

4.2 Register

POST /register

Authentication: Not required

Creates a new user account. Username and email must each be unique; password is hashed with bcrypt before being stored.

Request Body

Field	Type	Description
username	string	Required. Must not already exist.
email	string	Required. Must not already exist.
password	string	Required. Stored only as a bcrypt hash, never in plain text.

Responses

Status	Meaning	When it happens
201	Created	User inserted successfully. Returns { message, user_id }.
400	Bad Request	username, email, or password missing/empty.
409	Conflict	Username or email already exists.
500	Server Error	Unexpected database error; transaction is rolled back.

Example — 201 Created

```
{
  "message": "User registered successfully.",
  "user_id": 42
}
```

4.3 Login

POST /login

Authentication: Not required

Verifies credentials and issues a JWT valid for 1 hour.

Request Body

Field	Type	Description
username	string	Required.
password	string	Required. Compared against the stored bcrypt hash.

Responses

Status	Meaning	When it happens
200	OK	Credentials valid. Returns { token }.
401	Unauthorized	'Invalid username' if no user matches, or 'Invalid password' if the hash check fails.

Example — 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Decoded token payload:

```
{
  "user_id": 42,
  "username": "jane_doe",
  "exp": 1718049600
}
```

4.4 Predict — Article Text

POST `/predict/article`

Authentication: **Required — Bearer JWT token**

Analyzes pasted article text with the ISOT REAL/FAKE classifier and returns the full research dashboard (sentiment, readability, top keywords, related articles).

Request Body

Field	Type	Description
text	string	Required. Full article body. Must contain at least 10 words.

Responses

Status	Meaning	When it happens
200	OK	See Response Object Reference (section 5) — full prediction object with dashboard.
400	Bad Request	'text' missing/empty, or fewer than 10 words.
401	Unauthorized	Missing or invalid JWT (see section 3).

Internally calls `predict_and_respond(text, isot_model, isot_vectorizer, mode='article', labels={0:'FAKE',1:'REAL'}, run_dashboard=True)`. The verdict mapping is: label 0 → FAKE, label 1 → REAL.

4.5 Predict — From URL

POST `/predict/url`

Authentication: **Required — Bearer JWT token**

Fetches the article body from a URL using `trafilatura`, then runs the same analysis as `/predict/article`.

Request Body

Field	Type	Description
url	string	Required. A publicly accessible article URL.

Responses

Status	Meaning	When it happens
200	OK	Same response shape as <code>/predict/article</code> — see section 5.
400	Bad Request	'url' missing/empty, the fetch failed, or extracted text < 10 words.
401	Unauthorized	Missing or invalid JWT.
422	Unprocessable Entity	<code>trafilatura</code> could not extract article text — site may be paywalled or block scraping. Client should suggest pasting the text directly.

Example — 422 Unprocessable Entity

```
{
  "error": "Could not extract article text from this URL. The site may be
paywalled or block automated access. Try pasting the article text directly
instead."
}
```

4.6 Predict — Quote / Statement

POST /predict/quote

Authentication: Required — Bearer JWT token

Analyzes a short political statement using the LIAR model. **Note: this endpoint is currently non-functional in app.py — liar_model and liar_vectorizer are commented out at import time, so calling this route will raise a NameError. Re-enable the LIAR model loading lines before deploying this endpoint.**

Request Body

Field	Type	Description
text	string	Required. A short statement or quote. Must contain at least 3 words.

Responses (once LIAR model is enabled)

Status	Meaning	When it happens
200	OK	Response shape from section 5, without the analysis/related_articles dashboard fields (run_dashboard=False). verdict will be MISLEADING (label 0) or CREDIBLE (label 1).
400	Bad Request	'text' missing/empty, or fewer than 3 words.
401	Unauthorized	Missing or invalid JWT.
500	Server Error	Currently always — LIAR model variables are undefined.

4.7 Prediction History

GET /history

Authentication: Required — Bearer JWT token

Returns the authenticated user's most recent predictions, newest first.

Query Parameters

Parameter	Type	Description
limit	integer	Optional. Max rows to return. Default 20.
offset	integer	Optional. Pagination offset. Default 0.

Responses

Status	Meaning	When it happens
200	OK	Returns { history, limit, offset }. history is an array of prediction rows.
401	Unauthorized	Missing or invalid JWT.
500	Server Error	Database error; message returned in error field.

Example — 200 OK

```
{
  "history": [
    {
      "id": 17,
      "mode": "article",
      "input": "Data centres are expected to consume...",
      "verdict": "REAL",
      "confidence": 91.4,
      "timestamp": "2026-06-10T14:32:08Z"
    }
  ],
  "limit": 20,
  "offset": 0
}
```

Known issue: the SQL query in /history selects columns id, mode, input, verdict, confidence, timestamp from the predictions table, but the schema defined in db.py (section 6) defines this table with columns id, user_id, input_text, verdict, confidence, top_words, created_at — there is no mode, input, or timestamp column. This query will fail against the current schema. Align the column names (e.g. input_text → input, created_at → timestamp, and add a mode column) before this endpoint will work.

5. Response Object Reference

This is the full shape returned by `/predict/article` and `/predict/url` when `run_dashboard=True`. Fields marked optional depend on external services (Groq, NewsAPI) being configured.

Top-level fields

Field	Type	Description
<code>verdict</code>	string	"REAL" or "FAKE" (article endpoints).
<code>confidence</code>	number	Model confidence for the predicted class, as a percentage (0–100), rounded to 1 decimal.
<code>label</code>	integer	Raw model output: 0 or 1 (see label mapping per endpoint).
<code>top_words</code>	array	Top 5 TF-IDF features driving the prediction. See 'top_words item' below.
<code>mode</code>	string	Echoes the request type: "article" or "quote".
<code>analysis</code>	object	Linguistic dashboard. Only present when <code>run_dashboard=True</code> . See below.
<code>generated_title</code>	string	Short 5–7 word title generated by Groq (or TF-IDF fallback). Only present when <code>run_dashboard=True</code> .
<code>related_articles</code>	object	Related/verified sources from NewsAPI. Only present when <code>run_dashboard=True</code> .

top_words item

Field	Type	Description
<code>word</code>	string	The token/feature name.
<code>score</code>	number	TF-IDF score for this token in the input, rounded to 4 decimals.

analysis.sentiment

Field	Type	Description
<code>polarity</code>	number	TextBlob polarity, -1 (negative) to 1 (positive), rounded to 3 decimals.
<code>polarity_label</code>	string	"Positive" (>0.2), "Negative" (<-0.2), or "Neutral".
<code>subjectivity</code>	number	TextBlob subjectivity, 0 (objective) to 1 (subjective), rounded to 3 decimals.
<code>subjectivity_label</code>	string	"Highly opinionated" (>0.6), "Somewhat opinionated" (>0.4), or "Mostly objective".

analysis.readability

Field	Type	Description
<code>grade_level</code>	number	Flesch-Kincaid grade level (textstat), rounded to 1 decimal.

reading_ease	number	Flesch reading ease score (textstat), rounded to 1 decimal. Higher = easier to read.
--------------	--------	--

analysis.stats

Field	Type	Description
word_count	integer	Total words in the (cleaned) input text.
sentence_count	integer	Sentence count via textstat.
exclamation_count	integer	Count of '!' characters in the input.
caps_words	integer	Count of words that are fully uppercase and longer than 2 characters.

related_articles

Field	Type	Description
framing	string	Human-readable framing sentence. Differs based on verdict — see below.
articles	array	Up to 3 related articles from NewsAPI. See 'related article item' below.

framing text by verdict:

- FAKE → "Here is what verified sources are actually reporting on this topic:"
- REAL → "Related sources to deepen your research on this topic:"

related article item

Field	Type	Description
title	string	Article headline from NewsAPI.
source	string	Publication name (e.g. "Reuters").
url	string	Link to the original article.
published_at	string	Publication date, YYYY-MM-DD.
excerpt	string	Best-matching sentence from the article (highest keyword overlap with top_words), truncated to 300 chars.
citation	string	APA 7th-edition citation, auto-generated. See format note below.

APA citation format: Author, A. A. (Year, Month Day). Title. Publication. URL — or, if the author is missing/unusable (e.g. an email address), the publication name takes the author position: Publication. (Year, Month Day). Title. URL

Full example response — /predict/article (REAL)

```
{
  "verdict": "REAL",
  "confidence": 87.3,
  "label": 1,
  "mode": "article",
  "top_words": [
```

```
{ "word": "researchers", "score": 0.3142 },
{ "word": "centres", "score": 0.2890 },
{ "word": "report", "score": 0.2415 },
{ "word": "environmental", "score": 0.2100 },
{ "word": "institute", "score": 0.1873 }
],
"analysis": {
  "sentiment": {
    "polarity": -0.052,
    "polarity_label": "Neutral",
    "subjectivity": 0.214,
    "subjectivity_label": "Mostly objective"
  },
  "readability": {
    "grade_level": 12.4,
    "reading_ease": 41.2
  },
  "stats": {
    "word_count": 64,
    "sentence_count": 2,
    "exclamation_count": 0,
    "caps_words": 1
  }
},
"generated_title": "AI Data Centres Power and Water Demand",
"related_articles": {
  "framing": "Related sources to deepen your research on this topic:",
  "articles": [
    {
      "title": "AI boom fuels surge in data centre energy use",
      "source": "Reuters",
      "url": "https://www.reuters.com/...",
      "published_at": "2026-06-04",
      "excerpt": "The United Nations University Institute warned that data centres could double their water and power consumption by 2030.",
      "citation": "Reuters. (2026, June 4). AI boom fuels surge in data centre energy use. https://www.reuters.com/..."
    }
  ]
}
```

6. Database Schema (db.py)

PostgreSQL access is managed through a ThreadedConnectionPool (1–10 connections). `init_pool()` creates the pool and `init_schema()` runs the CREATE TABLE IF NOT EXISTS statements below — both run once at startup.

users

Column	Type	Notes
id	SERIAL PK	Auto-incrementing primary key.
username	VARCHAR(50)	Unique, not null.
email	VARCHAR(255)	Unique, not null.
password_hash	TEXT	bcrypt hash. Never store plain text passwords.
created_at	TIMESTAMP	Defaults to NOW().

predictions

Every article analysis is logged here. `user_id` is nullable to allow anonymous predictions; FK uses ON DELETE SET NULL so a deleted user's predictions are retained but anonymized.

Column	Type	Notes
id	SERIAL PK	Auto-incrementing primary key.
user_id	INTEGER FK → users(id)	Nullable. ON DELETE SET NULL.
input_text	TEXT	Not null. Truncated to 2000 chars on insert (log_prediction).
verdict	VARCHAR(10)	e.g. "REAL" / "FAKE".
confidence	REAL	Model confidence percentage.
top_words	JSONB	Top TF-IDF words, stored as JSON for future querying.
created_at	TIMESTAMP	Defaults to NOW().

saved_articles

Lets a user bookmark one of the related/suggested articles returned by a prediction.

Column	Type	Notes
id	SERIAL PK	Auto-incrementing primary key.
user_id	INTEGER FK → users(id)	Not null. ON DELETE CASCADE.
prediction_id	INTEGER FK → predictions(id)	Nullable. ON DELETE SET NULL.
title	TEXT	Not null.

snippet	TEXT	Article excerpt.
similarity	REAL	Similarity score to the original input.
overlap	INTEGER	Keyword overlap count.
saved_at	TIMESTAMP	Defaults to NOW().

Indexes

- `idx_predictions_user` on `predictions(user_id)` — speeds up history lookups.
- `idx_saved_user` on `saved_articles(user_id)`
- `idx_saved_prediction` on `saved_articles(prediction_id)`

6.1 Query Helper Functions

`db.py` exposes thin wrapper functions so `app.py` never touches raw `psycopg2` cursors directly. Each helper acquires a connection from the pool, performs the query, and releases the connection in a finally block.

Function	Returns	Description
<code>create_user(username, email, password_hash)</code>	→ int	Inserts a user, returns new id.
<code>get_user_by_email(email)</code>	→ dict None	Returns { id, username, email, password_hash }.
<code>get_user_by_id(user_id)</code>	→ dict None	Returns { id, username, email } (no password hash).
<code>log_prediction(user_id, input_text, verdict, confidence, top_words)</code>	→ int	Inserts a prediction (text truncated to 2000 chars), returns new id.
<code>get_user_predictions(user_id, limit=50)</code>	→ list[dict]	Most recent predictions, input_text truncated to 300 chars, created_at as ISO string.
<code>save_article(user_id, prediction_id, title, snippet, similarity, overlap)</code>	→ int	Inserts a saved article, returns new id.
<code>get_saved_articles(user_id)</code>	→ list[dict]	All saved articles for the user, newest first.
<code>delete_saved_article(user_id, saved_article_id)</code>	→ bool	Deletes if owned by user. Returns whether a row was deleted.
<code>delete_prediction(user_id, prediction_id)</code>	→ bool	Deletes if owned by user. Returns whether a row was deleted.

Note: `app.py`'s `/history` route and `/register` and `/login` routes use a separate ad-hoc connection via `get_db_connection()` (raw `psycopg2.connect` with `RealDictCursor`) rather than the pooled helpers in `db.py`. This is inconsistent with the rest of the app and bypasses the connection pool — consider migrating these routes to use `get_conn()/release_conn()` or the `db.py` helper functions (e.g. `create_user`, `get_user_predictions`) for consistency and to avoid exhausting database connections under load.

7. Error Handling Conventions

All error responses follow the same shape:

```
{
  "error": "Human-readable description of what went wrong."
}
```

Common status codes across the API

Status	Meaning	When it happens
200	OK	Request succeeded.
201	Created	Resource created (register).
400	Bad Request	Missing/empty required fields, or input too short (article < 10 words, quote < 3 words).
401	Unauthorized	Missing/invalid JWT, or invalid login credentials.
409	Conflict	Duplicate username or email on registration.
422	Unprocessable Entity	URL fetch succeeded but no article text could be extracted.
500	Server Error	Unexpected exception — database error, unhandled model error, etc.

Operational notes

- CORS is enabled globally via `flask_cors.CORS(app)` — all origins are currently allowed.
 - The connection pool is initialized twice in `app.py` — once inside the `db-availability` `try/except` block (which sets `USE_DB`), and again at the bottom of the file before `app.run()`. This is redundant but not harmful since `ThreadedConnectionPool` re-initialization just replaces the global pool object.
 - Predictions are logged best-effort: if the database is unavailable (`USE_DB = False`) or `log_prediction` raises, the error is printed and the request still returns a 200 with the prediction result.
 - The `/predict/quote` endpoint references `liar_model` and `liar_vectorizer`, which are commented out at the top of `app.py` — this route is currently broken and will throw a `NameError` if called. Uncomment the LIAR model loading lines and ensure `liar_model.pkl` exists in the `models` directory to enable it.
-